

SECTION 1: DATA DICTIONARY (DDIC)

Q1. What is the ABAP Data Dictionary (DDIC)? What are its key components?

Answer:

The ABAP Data Dictionary (DDIC) is a central repository in SAP that manages all metadata/definitions for database objects used across SAP applications. It acts as a single source of truth for data structures – any change in DDIC automatically propagates to all programs using those objects.

Key Components:

- **Tables:** Transparent (1:1 mapping with DB table), Pooled (multiple logical tables in one physical pool), Cluster (stores data in compressed cluster key).
- **Views:** Database View (join at DB level), Projection View (subset of fields), Help View (for F4 help), Maintenance View (for SM30 maintenance), Billing View
- **Structures:** Flat structures (no data stored in DB) used for data transfer/interface
- **Data Elements:** Defines the business meaning of a field (domain, field label, documentation, parameter ID)
- **Domains:** Defines technical attributes – data type (CHAR, INT, DATS, etc.), length, value ranges
- **Search Helps:** Provides F4 input help – Elementary (single table) and Collective (multiple search paths)
- **Lock Objects:** Define logical locks for concurrent data access using ENQUEUE/DEQUEUE
- **Type Groups:** Group of TYPE definitions reusable across programs

Hierarchy: Domain → Data Element → Table Field / Structure Field

Q2. Explain the difference between Transparent Tables, Pooled Tables, and Cluster Tables.

Answer:

Transparent Tables:

- 1:1 relationship with the physical database table.
- Each field in DDIC corresponds to exactly one column in the database.
- Directly accessible via native SQL. Most common type used in ABAP development.
- Example: MARA (Material Master), KNA1 (Customer Master)

Pooled Tables:

- Multiple pooled tables are stored in a single physical DB table called a 'Table Pool'.
- Used for small configuration/customizing data tables that have many records but few entries per table.
- Access only via Open SQL; no native SQL access. Cannot be joined at DB level.
- Example: TRDIR (program attributes), TSTC (transaction codes)

Cluster Tables:

- Multiple cluster tables are stored in a single physical DB table called a 'Table Cluster'.
- Stores variable-length data in a compressed VARDATA field. Key fields identify which records belong to which table.
- Used for data with long, variable content like documents, long text. Worse performance for large data.
- Example: CDPOS (change document items), PCL1/PCL2 (HR payroll clusters)

Key Difference Summary:

Transparent: Direct DB table. Pooled: Many tables, one pool DB table.
Cluster: Complex compressed storage in cluster table.
For HANA, pooled and cluster tables are migrated to transparent tables.

Q3. What are the different types of Views in DDIC and when would you use each?

Answer:

- **Database View:** Performs JOIN at database level on multiple transparent tables with linked foreign key relationships. Read-only (mostly). Best performance since join runs in DB.
- **Projection View:** Restricts fields/rows of a single transparent table. Hides sensitive columns. Read-only.
- **Help View:** Used as basis for Search Help (F4 help). Can join multiple tables. Provides selection options at runtime.
- **Maintenance View:** Used for table maintenance via SM30. Allows VIEW MAINTENANCE to maintain related tables together as a logical unit. Used in IMG (configuration).
- **External/Remote Views (CDS-based):** In modern ABAP/HANA, CDS views replace most traditional views for analytics and Fiori.

When to use which:

Database View → Performance-critical reporting joins.

Projection View → Restrict sensitive data.

Help View → F4 search help.

Maintenance View → Customizing data maintenance via SM30.

Q4. What is a Search Help? Differentiate between Elementary and Collective Search Helps.

Answer:

A Search Help (F4 Help) is a DDIC object that provides a list of possible input values (a value selection list) for a screen field when the user presses F4 or clicks the dropdown button.

Elementary Search Help:

- Searches a single data source (table, view, or function module result).
- Has a single selection method and a single set of return fields.
- Configured with: Selection method (table/view), Dialog type (display all/restrict by condition), Search help exit (optional FM to modify behavior), Import/Export parameters (screen fields mapped to search help fields).

Collective Search Help:

- Groups multiple Elementary Search Helps under a single help object.
- The user can choose between different search paths (tabs in the F4 popup).
- Useful when data can be searched by different criteria (e.g., search material by number OR by description OR by old material number).

Search Help Exit:

A function module attached to a search help to add custom logic – filter values, call RFC, preselect entries, etc. Must follow a specific interface (SHLP_DESCR_T, CALLCONTROL, etc.).

Q5. What are Lock Objects in ABAP? How do you implement locking in SAP?

Answer:

Lock Objects in DDIC define logical locks to control concurrent access to SAP data. Unlike database locks (row-level, short-lived), SAP logical locks are maintained in the lock table (DDIC) and managed by the Enqueue server.

How Locking Works:

- **ENQUEUE-<lockobject>:** Sets (acquires) the lock. Returns exception if already locked.
- **DEQUEUE-<lockobject>:** Releases the lock explicitly.
- Locks are automatically released at end of LUW (Logical Unit of Work) or user session end.

Lock Modes:

- **S (Shared):** Multiple users can read simultaneously. No write allowed.
- **E (Exclusive):** Only one user can access. Others blocked for read and write.
- **X (Exclusive Non-cumulative):** Like E but cannot be accumulated by the same user.

Implementation Example:

```
CALL FUNCTION 'ENQUEUE_EZMATERIAL'
  EXPORTING matnr = lv_matnr
  EXCEPTIONS foreign_lock = 1 already_locked = 2.
IF sy-subrc <> 0.
  MESSAGE 'Record locked' TYPE 'E'.
ENDIF.
```

Best Practice:

Always use a corresponding DEQUEUE call in a CLEANUP or after successful COMMIT WORK. In module pools, set locks on PAI of the first screen and release on SAVE or BACK.

SECTION 2: RICEWF (REPORTS, INTERFACES, CONVERSIONS, ENHANCEMENTS, WORKFLOWS, FORMS)

Q6. What does RICEWF stand for? Explain each component briefly.

Answer:

- **R – Reports:** Output programs (ALV reports, classical reports). Extract and display data. Examples: SE38 report with ALV grid.
- **I – Interfaces:** Programs that exchange data between SAP and external systems. Types: IDocs, BAPIs, RFCs, Web Services, File Interfaces (SFTP/FTP).
- **C – Conversions:** Data migration programs using BDC (Batch Data Communication), LSMW, BAPI, or Direct Input methods to load legacy data into SAP.
- **E – Enhancements:** Custom code added to standard SAP without modifying it. Types: User Exits, BAdIs, Enhancement Spots, Implicit/Explicit Enhancement Points.
- **W – Workflows:** Business process automation using SAP Business Workflow / SAP WS. Triggers automated steps, notifications, approvals based on business events.
- **F – Forms:** Printed output documents. Types: SAPScript (old), SmartForms (medium), Adobe Forms (current standard), SAP XSLT/Groovy.

MNC Context:

In MNC interviews, you'll often be asked which RICEWF objects you've worked on, or, be ready with specific examples: 'I built a BDC-based conversion for Material Master (MM01) using CALL TRANASCTION and created ALV grid reports for inventory analysis.'

SAP ABAP & ABAP on HANA - INTERVIEW PREPARATION GUIDE

50 QUESTIONS | BASICS TO 4 YEARS EXPERIENCE

Topics: Data Dictionary • RICEWF • Module Pools • OOP ABAP • CDS • AMDP • Performance Tuning • ABAP 7.5 Syntax

TOP MNC PREPARATION
2024 EDITION

Q7. Explain BDC (Batch Data Communication). What are the different BDC techniques?

Answer:

BDC (Batch Data Communication) is an SAP data migration technique that simulates foreground SAP transaction screen inputs in the background, allowing bulk data upload from legacy systems.

BDC Techniques:

1. **Call Transaction Method:** Calls a transaction synchronously. Each record is processed immediately. Errors can be handled record-by-record. Faster for small-medium volumes.
2. **Session Method (RSBDCSUB):** Records are added to a BDC session. Transactions are processed asynchronously via SM35. Better for large volumes. Can run in background.
3. **Direct Input Method:** Uses special function modules that bypass screen processing. Fastest method. Only available for specific standard objects (Materials via RMDATIND, GL via RFBIBLOO).

BDC Structure (BDCDATA):

BDCDATA: PROGRAM, DYNPRO (screen number), DYNBEGIN ('X'), FNAM (field name), FVAL (value).

Call Transaction Example:

```
CALL TRANSACTION 'MM01' USING t_bdcdata MODE 'N'  
UPDATE 'S' MESSAGES INTO t_messtab.
```

Modes:

- A = Foreground (shows screens)
- E = Background (errors shown in foreground)
- N = Background (no screen display)

Error Handling:

Use MESSAGE table (BDCMSGCOLL) to capture errors. Log failed records to a custom error table and provide a reprocessing report.

Q8. What is an IDoc? Explain IDoc architecture and its key segments.

Answer:

An IDoc (Intermediate Document) is a standard SAP data structure for electronic data interchange between SAP and external systems (EDI, SAP-to-SAP). It is a container that holds business data in a hierarchical segment structure.

IDoc Architecture:

1. **Control Record (EDIDC):** Contains metadata - IDoc number, message type, direction (inbound/outbound), partner number, port, status, creation date.
2. **Data Records (EDID4):** Actual business data. Each segment is one data record. SEGNAME = segment name, SDATA = 1000-byte data field.
3. **Status Records (EDIDS):** Log of all processing steps and status changes (e.g., 01=IDoc generated, 03=passed to port, 12=dispatch OK, 53=application document posted).

Key Objects:

- **Message Type:** Business meaning (ORDERS = Purchase Order, INVOIC = Invoice, MATMAS = Material Master).
- **Basic Type (IDoc Type):** Technical structure definition (e.g., ORDERS05).
- **Extension Type:** Customer-specific additional segments appended to basic type.
- **Port:** Defines the communication channel (file, RFC, HTTP).
- **Partner Profile (WE20):** Defines processing parameters per trading partner - message type, direction, processing mode.

Inbound vs Outbound:

- **Outbound:** SAP generates IDoc → Partner profile → Port → Sent to external system.
- **Inbound:** External system sends IDoc → Port → WE20 → Inbound function module posts to SAP.

Transaction Codes:

WE02/WE05 (IDoc List), WE20 (Partner Profiles), WE30 (IDoc Type), BD10-BD12 (send master data), WE19 (test IDoc), BD87 (inbound reprocessing).

Q9. What are the different types of Enhancements in SAP ABAP? How do BADIs work?

Answer:

Types of SAP Enhancements (Evolution):

1. **User Exits:** Older SAP-provided subroutines (FORM/FUNCTION) in standard programs where custom code can be added. Find via SMOD/CMOD.
2. **Customer Exits:** Function module exits (EXIT_<program>_<NNN>), Menu exits, Screen exits. Managed via SMOD.
3. **BADIs (Business Add-Ins):** OOP-based enhancement technology. Standard code calls defined interface methods. You implement those methods with custom logic. Managed via SE18/SE19.
4. **Enhancement Spots / Enhancement Sections (ABAP 7.0+):** Explicit and Implicit enhancement points that allow inserting code at specific places in standard programs without modifying them.
5. **Kernel BADIs (New BADIs):** Use enhancement spots instead of separate BADI objects. Support multiple implementations, filter-based activation, instance-based.

How BADIs Work:

- **Definition (SE18):** SAP defines a BADI with an interface (ZIF_BADI...). Specifies methods to be implemented.
- **Implementation (SE19):** You create a BADI implementation for a specific BADI definition. Create a class that implements the BADI interface and add custom logic in the methods.
- **Multiple Implementations:** Multiple BADIs can be active simultaneously. SAP calls all active implementations. Use 'filter-based' BADIs to activate only for specific criteria.

Code to call a Classic BADI:

```
DATA: lo_badi TYPE REF TO badi_name.  
GET BADI lo_badi.  
CALL BADI lo_badi->method_name EXPORTING .... IMPORTING ....
```

New BADI (Enhancement Spot):

```
DATA(lo_badi) = cl_badi_query=>get_badi( 'BADI_NAME' ).  
CALL BADI lo_badi->method_name(...).
```

Q10. Explain SAP SmartForms vs Adobe Forms. When would you choose one over the other?

Answer:

SmartForms:

- SAP proprietary form tool. Designed as a replacement for SAPScript. Uses a graphical form builder with a tree structure.
- Logic is embedded in the form itself (via nodes: text, loop, table, condition, command nodes).
- Generated as a function module at activation. Called via CALL FUNCTION 'SSF_FUNCTION_MODULE_NAME' then calling that FM.
- Supports multiple pages, windows, background graphics. Output via spool (print, PDF, fax).
- Limitation: Complex layouts difficult to design; not interactive; limited PDF form filling.

Adobe Forms (Interactive / Static):

- Based on Adobe LiveCycle Designer. SAP integrated via ADS (Adobe Document Services).
- Two types: Static (fixed layout, non-editable PDF output) and Interactive (editable PDF fields, digital signatures, submit via WebDynpro/Fiori).
- Much better for complex, print-quality layouts. Full PDF capabilities including encryption, digital signatures.
- Called via FP_FUNCTION_MODULE_NAME + generated FM call or via FP_JOB_OPEN/CLOSE pattern.

Comparison:

- **Use SmartForms:** Legacy systems, simpler print forms, no ADS server available, less complex layout.
- **Use Adobe Forms:** Complex layout requirements, interactive PDF (approval forms, GRN, invoice attachments), Fiori integration, digital signatures, barcode support.

MNC Note: In most recent MNC SAP projects, Adobe Forms is the standard for all new form development. SmartForms knowledge is needed for maintenance of older forms.

SECTION 3: MODULE POOL PROGRAMS

Q11. What is a Module Pool? How is it different from a Report program?

Answer:

A Module Pool (also called Dialog Program) is an ABAP program type (type M) used to create custom screen-based interactive transactions. Unlike reports that run top-to-bottom, Module Pools are event-driven - the program responds to user actions on screens.

Key Differences:

- **Module Pool:** Type M, requires screens (Dynpro), transaction code, no START-OF-SELECTION, uses PBO/PAI flow logic, multiple screens possible.
- **Report:** Type S (executable), runs linearly from START-OF-SELECTION, uses selection screens, outputs via WRITE or ALV.

Components of a Module Pool:

1. **Main Program (Type M):** Contains global data declarations, subroutines, MODULE implementations.
2. **Screens (Dynpros):** Created in Screen Painter (SE51). Each has a Screen Number, Attributes, Layout (Fields, buttons), Flow Logic (PBO/PAI).
3. **GUI Status:** Menu bar, toolbar, function key assignments created in Menu Painter (SE41).
4. **Transaction Code:** Entry point for end users (SE93).

Flow Logic Events:

- **PBO (Process Before Output):** Executes before screen is displayed. Used to populate screen fields, set field attributes, set GUI status.
- **PAI (Process After Input):** Executes after user action (press Enter, click button). Used to validate input, call business logic, navigate to next screen.
- **POH (Process On Help Request):** Called when F1 pressed on a field.
- **POV (Process On Value Request):** Called when F4 pressed (popup F4 help).

Q12. Explain screen navigation and CALL SCREEN vs LEAVE TO SCREEN vs SET SCREEN in Module Pools.

Answer:

SET SCREEN <n>:

- Schedules the next screen to be displayed after the current PAI completes. Does not immediately navigate.
- Control returns to the rest of the current PAI module first, then after PBO of the new screen runs.

```
SET SCREEN 200. "Next screen will be 200 after PAI completes
```

CALL SCREEN <n>:

- Opens a new screen in a modal dialog box (sub-screen). Creates a new call stack level.
- Execution resumes after CALL SCREEN when the modal dialog is closed (LEAVE SCREEN in PAI of called screen).
- Used for popup-style detail screens. The calling screen is suspended, not closed.

```
CALL SCREEN 300 STARTING AT 10 S ENDING AT 70 20.  
"Popup at position
```

LEAVE TO SCREEN <n>:

- Immediately leaves the current screen and goes to screen n. Used to skip screens or go to a specific screen unconditionally.
- Use LEAVE TO SCREEN 0 to exit the module pool transaction completely.

```
LEAVE TO SCREEN 0. "Exit the transaction
```

LEAVE SCREEN:

- Leaves the current screen immediately (goes to the screen set by SET SCREEN or ends the call if 0).

```
"In PAI of Screen 120"
```

Q13. What is a Tabstrip and Subscreen in Module Pools? How are they implemented?

Answer:

Tabstrip:

A Tabstrip is a UI control that displays multiple tabs on a screen. Each tab contains a subscreen (an embedded screen with its own layout). Used to organize complex data into logical groups (e.g., General Data, Sales Data, Purchasing Data tabs).

Implementation Steps:

1. In Screen Painter, drag a 'Tabstrip' control onto the main screen. Give it a name (e.g., TC_TABS).
2. Add Tab pages and define a Subscreen Area within each tab.
3. Create separate Subscreen screens (e.g., 110, 120, 130) for each tab's content.
4. In PBO, set the current active tab and call the subscreen:

```
MODULE set_tab OUTPUT.  
tc_tabs-activetab = 'TAB1'.  
tc_tabs-dynnr = '0150'.  
ENDMODULE.
```

5. In Flow Logic, use CALL SUBSCREEN for each subscreen area:
PBO: CALL SUBSCREEN sub_area INCLUDING sy-repid g_dynnr
PAI: CALL SUBSCREEN sub_area

Subscreen (without Tabstrip):

A Subscreen is a screen type (set attribute = 'S') that can be embedded into another screen. Used to reuse common UI sections across multiple screens. The container screen uses CALL SUBSCREEN in its flow logic.

SAP ABAP & ABAP on HANA - INTERVIEW PREPARATION GUIDE

MNC READY

50 QUESTIONS | BASICS TO 4 YEARS EXPERIENCE | TOP MNC PREPARATION | 2024 EDITION

Topics: Data Dictionary • RICEWF • Module Pools • OOP ABAP • CDS • AMDP • Performance Tuning • ABAP 7.5 Syntax

SECTION 4: OBJECT-ORIENTED ABAP (OOP ABAP)

Q14. Explain the key OOP concepts in ABAP with examples.

1. Encapsulation:

Bundling data (attributes) and methods together in a class.
Visibility controls access.

- **PUBLIC** : Accessible from outside the class.
- **PROTECTED** : Accessible within the class and its subclasses.
- **PRIVATE** : Accessible only within the class itself.

CLASS
Attributes
Methods

2. Inheritance:

```
CLASS zcl_vehicle DEFINITION.  
  PUBLIC SECTION.  
  METHODS: drive.  
ENDCLASS.
```

→ Parent

```
CLASS zcl_car DEFINITION INHERITING FROM zcl_vehicle.  
  PUBLIC SECTION.  
  METHODS: honk.  
ENDCLASS.
```

→ Child

3. Polymorphism:

A method defined in a superclass can be overridden in subclasses.
Call the method via a superclass reference - the correct subclass method executes at runtime.

```
DATA: lo_v TYPE REF TO zcl_vehicle.  
lo_v = NEW zcl_car().  
lo_v->drive(). "Calls zcl_car's drive if overridden
```

4. Abstraction:

Abstract classes (ABSTRACT keyword) define templates with methods that must be implemented by subclasses. Cannot be instantiated.

5. Interfaces:

Defines a contract (method signatures) that implementing classes must fulfill. A class can implement multiple interfaces.

```
INTERFACE zif_printable.  
  METHODS: print.
```

```
CLASS zcl_doc DEFINITION.  
  PUBLIC SECTION.  
  INTERFACES: zif_printable.  
ENDCLASS.
```

```
CLASS zcl_doc IMPLEMENTATION.  
  METHOD zif_printable~print.  
    WRITE "Printing".  
  ENDMETHOD.  
ENDCLASS.
```

KEY TAKEAWAY

OOP brings reusability, maintainability and scalability to ABAP development. ☆

Q17. Explain Exception Handling in ABAP OOP. Difference between CX_STATIC_CHECK, CX_DYNAMIC_CHECK, CX_NO_CHECK?

Answer:

OOP ABAP uses class-based exceptions (CLASS-BASED EXCEPTIONS) instead of sy-subrc checks.

Exception Class Hierarchy

- **CX_ROOT** : Root of all exception classes.
- **CX_STATIC_CHECK** : Must be declared in RAISING clause and handled with TRY-CATCH. Checked at compile time. Used for expected, recoverable errors.
- **CX_DYNAMIC_CHECK** : Should be declared in RAISING but compiler does not force. Checked at runtime. Used for programming errors.
- **CX_NO_CHECK** : Never needs to be declared. Used for severe system errors (e.g., memory, stack overflow).

Raising an Exception:

```
RAISE EXCEPTION TYPE zcx_my_exception  
EXPORTING textid = zcx_my_exception=>invalid_input  
          param1 = lv_value.
```

Catching Exceptions:

```
TRY.  
  lo_obj->risky_method().  
CATCH zcx_specific_error INTO DATA(lx_err).  
  MESSAGE lx_err->get_text() TYPE 'E'.  
CATCH cx_root INTO DATA(lx_root).  
  " Catch-all for unexpected exceptions  
ENDTRY.
```

TIP

Always catch CX_ROOT as last catch to avoid unhandled exceptions. ☆

Creating Custom Exception Class:

Use SE24, create class inheriting from CX_STATIC_CHECK. Add exception texts in Texts tab. Use TEXTID attribute for identifying specific error scenarios.

Q15. What is the difference between a Class and an Interface in ABAP? When would you use each?

CLASS

- Blueprint that defines attributes (data) and methods (behavior). Can have implementation code.
- Supports single inheritance only.
- Can be Abstract or Final.
- Has constructor (CONSTRUCTOR for instance, CLASS_CONSTRUCTOR for static/class level).

INTERFACE

- Defines only method signatures (no implementation).
- A class can implement multiple interfaces.
- Access via "~" notation:
lo_obj->zif_print~print().
- Useful for contracts: BADIs, ALV events, factory patterns.

WHEN TO USE?

- Use **CLASS** : When you have shared implementation and need inheritance.
- Use **INTERFACE** : When you want a contract across different implementations (loose coupling, testability).

REAL SAP EXAMPLES

- **IF_SALV_EVENTS_TABLE** : Interface for ALV event handling.
- **IF_AMDP_MARKER_HDB** : Marker interface to declare an AMDP class.

Q16. What are static (class) vs instance methods and attributes? Explain with examples.

INSTANCE (OBJECT) MEMBERS

- Belong to a specific object instance. Each object has its own copy.
- Must create an object to call.
- Declared without CLASS keyword.

```
CLASS zcl_person DEFINITION.  
  PUBLIC SECTION.  
  METHODS: get_name.  
  PRIVATE SECTION.  
  DATA: mv_name TYPE string.  
ENDCLASS.
```

```
DATA(lo_person) = NEW zcl_person().  
lo_person->get_name().
```

STATIC (CLASS) MEMBERS

- Belong to the class itself, shared across all instances.
- Exist without creating an object.
- Declared with CLASS keyword.

```
CLASS zcl_util DEFINITION.  
  PUBLIC SECTION.  
  CLASS-METHODS: get_count.  
  PRIVATE SECTION.  
  CLASS-DATA: gv_count TYPE i.  
ENDCLASS.
```

```
zcl_util=>get_count().  
WRITE zcl_util=>gv_count.
```

PRACTICAL EXAMPLE - SINGLETON PATTERN

```
CLASS zcl_singleton DEFINITION.  
  PUBLIC SECTION.  
  CLASS-METHODS: get_instance RETURNING VALUE(ro_inst) TYPE REF TO zcl_singleton.  
  PRIVATE SECTION.  
  CLASS-DATA: go_instance TYPE REF TO zcl_singleton.  
ENDCLASS.  
CLASS zcl_singleton IMPLEMENTATION.  
  METHOD get_instance.  
    IF go_instance IS INITIAL.  
      go_instance = NEW zcl_singleton().  
    ENDIF.  
    ro_inst = go_instance.  
  ENDMETHOD.  
ENDCLASS.
```

RULE OF THUMB

Use static members for utilities, counters, singletons, factories. Use instance members for object-specific state & behavior. ☆

Q18. What is Design Pattern? Explain Factory Pattern and Strategy Pattern in ABAP OOP.

Answer:

Design Patterns are reusable solutions to common software design problems. They help build maintainable, extensible enterprise applications.

FACTORY PATTERN

Creates objects without exposing the instantiation logic. Factory decides which subclass to create.

Factory Class

```
CLASS zcl_output_factory DEFINITION.  
  PUBLIC SECTION.  
  CLASS-METHODS: create IMPORTING iv_type TYPE c  
    RETURNING VALUE(ro_obj) TYPE REF TO zif_output.  
ENDCLASS.
```

```
METHOD create.  
  CASE iv_type.  
    WHEN 'PDF'.  
      ro_obj = NEW zcl_pdf_output().  
    WHEN 'EXCEL'.  
      ro_obj = NEW zcl_excel_output().  
  ENDCASE.  
ENDMETHOD.
```

STRATEGY PATTERN

Defines a family of algorithms, encapsulates each one, and makes them interchangeable. Algorithm can be switched at runtime.

Interface for strategies

```
INTERFACE zif_calc_strategy.  
  METHODS: calculate IMPORTING iv_amount TYPE p  
    RETURNING VALUE(rv_result) TYPE p.  
ENDINTERFACE.
```

Concrete strategies (implement calculate).
- zcl_tax_inclusive
- zcl_tax_exclusive

Context class

- DATA: mo_strategy TYPE REF TO zif_calc_strategy.
- SET_STRATEGY method sets mo_strategy.
- execute calls mo_strategy->calculate().

OTHER IMPORTANT PATTERNS

- **Singleton** : One instance of a class per session.
- **Observer** : Event-driven communication (events).
- **Template Method** : Define algorithm skeleton in base class, defer steps to subclasses. ☆

SAP ABAP & ABAP on HANA - INTERVIEW PREPARATION GUIDE

MNC READY
2024 EDITION

50 QUESTIONS | BASICS TO 4 YEARS EXPERIENCE | TOP MNC PREPARATION | 2024 EDITION

Topics: Data Dictionary • RICEWF • Module Pools • OOP ABAP • CDS • AMDP • Performance Tuning • ABAP 7.5 Syntax

SECTION 5: CORE DATA SERVICES (CDS VIEWS)

Q19. What is a CDS View? How is it different from a traditional DDIC View?

Answer:

CDS (Core Data Services) is a data modeling infrastructure that allows defining semantically rich data models using an SQL-like DDL. CDS Views are defined in ADT (Eclipse) and reside in the ABAP repository.

CDS View Advantages over DDIC Views:

- **Annotations:** CDS supports metadata annotations (@AbapCatalog, @Analytics, @OData, @UI, @Semantics) for Fiori, OData, Analytics - all in one place.
- **Push-down to HANA:** Executed natively on HANA DB engine. Joins & aggregations run at DB level, not application server.
- **Association (Lazy Join):** Associations are executed only when fields are requested. DDIC views always join all tables.
- **Expressions:** Supports CASE, arithmetic, string functions, CAST, COALESCE, CURRENCY_CONVERSION - not in DDIC views.
- **Input Parameters:** CDS views can accept input parameters at runtime.
- **Inheritance/Extension:** Extend via EXTEND VIEW without modifying original view.

Simple CDS View Example:

```
@AbapCatalog.sqlViewName: 'ZMAT_VIEW'
@AbapCatalog.compiler.compareFilter: true
define view ZCDS_MATERIAL as select from mara
  association [0..*] to marc as _plant
  on $projection.matnr = _plant.matnr
{
  key mara.matnr,
  mara.mtart,
  mara.meins,
  _plant
}
```

CDS View Types:

- **Basic Interface View (suffix _I):** Selects from DB tables.
- **Composite Interface View:** Selects from other CDS views, adds joins/enrichment.
- **Consumption View (Fiori):** Annotations for UI layer (@UI.LineItem, @UI.selectionField).
- **Transactional Views (RAP):** Root/Projected views for BO interfaces.

Q20. What are CDS Annotations? Explain key annotations used in Fiori/OData development.

Answer:

CDS Annotations are metadata decorators (starting with @) applied to CDS views, fields, and associations to define behavior across SAP stack.

Key Annotation Categories:

1. **@AbapCatalog Annotations:**
 - @AbapCatalog.sqlViewName: 'ZSQLVIEW' - Mandatory.
 - @AbapCatalog.compiler.compareFilter: true - Enables filter pushdown.
2. **@Analytics Annotations:**
 - @Analytics.dataCategory: #CUBE - Analytical cube.
 - @Analytics.dataCategory: #DIMENSION - Dimension data.
 - @Semantics.amount.currencyCode: 'WAERS' - Currency code for amount.
3. **@OData Annotations:**
 - @OData.publish: true - Auto-publishes as OData V2 service.
 - @OData.entityType.name: 'Material' - Defines entity type name.
4. **@UI Annotations (Fiori Elements):**
 - @UI.LineItem: [{ position: 10, label: 'Material' }] - List report columns.
 - @UI.selectionField: [{ position: 10 }] - Adds to filter bar.
 - @UI.headerInfo: { typeName: 'Material', typeNamePlural: 'Materials', title.value: 'Matnr' } - Object page header.
 - @UI.FieldGroup: [{ qualifier: 'GENERAL', position: 10 }] - Group fields.
5. **@Search / @Semantics:**
 - @Search.searchable: true - Makes entity searchable.
 - @Semantics.text: true - Marks field as text for an ID.

Q21. What are CDS Associations and how do they differ from JOINS in CDS views?

Answer:

JOINS in CDS (Eager)

Traditional SQL JOIN - always executes regardless of whether fields are selected.

```
define view Z_SALES as select from vbak
  inner join vbap on vbak.vbeln = vbap.vbeln
{ vbak.vbeln, vbap.posnr, vbap.matnr }
```

Associations (Lazy/On-Demand)

Join path executed only when associated fields are used.

```
define view Z_SALES as select from vbak
  association [0..*] to vbap as _item
  on $projection.vbeln = _item.vbeln
  association [1..1] to knal as _customer
  on $projection.kunnr = _customer.kunnr
{ key vbak.vbeln, vbak.kunnr,
  _item, _customer }
  'Exposed as path elements'
```

Cardinality:

- [1..1] - Exactly one (1:1)
- [0..1] - Zero or one (optional 1:1)
- [0..*] - Zero or more (1:N)
- [1..*] - One or more

Using Associations:

- In CDS: SELECT FROM Z_SALES { vbeln, _customer.name }
- - joins customer only.
- In ABAP: SELECT ... FROM z_sales\item\matnr
- - path navigation (7.5+).

Best Practice: Use associations for optional/conditional relationships (better performance). Use JOINS when you always need the data. Expose associations with underscore prefix (_entity).

SECTION 6: ABAP MANAGED DATABASE PROCEDURES (AMDP)

Q22. What is AMDP? When and why would you use it over ABAP Open SQL?

Answer:

AMDP (ABAP Managed Database Procedures) allows writing HANA SQLScript procedures/functions inside ABAP classes and executes them on HANA DB engine.

How AMDP Works:

- Class implements marker interface IF_AMDP_MARKER_HDB.
- Methods implemented using BY DATABASE PROCEDURE/FUNCTION.
- SQLScript runs entirely on HANA - not on application server.
- ABAP calls method like normal class method; framework handles execution.

AMDP Procedure Example:

```
CLASS zcl_amdp_example DEFINITION PUBLIC.
  PUBLIC SECTION.
    INTERFACES if_amdp_marker_hdb.
  METHODS: get_top_materials IMPORTING VALUE(iv_count) TYPE i
    EXPORTING VALUE(et_result) TYPE tt_result.
ENDCLASS.
```

```
CLASS zcl_amdp_example IMPLEMENTATION.
  METHOD get_top_materials BY DATABASE PROCEDURE FOR HDB
    LANGUAGE SQLSCRIPT USING mara marc.
    et_result = SELECT TOP :iv_count matnr, mtart FROM mara
      ORDER BY matnr;
ENDMETHOD.
```

When to Use AMDP:

- Complex analytical calculations (CE functions, window functions, CTEs).
- Imperative SQLScript logic with loops, conditions, multiple interim tables.
- Performance-critical logic - heavy computation pushed to DB layer.

AMDP vs CDS:

	CDS	AMDP
Open SQL	Simple CRUD, standard business logic (DB-agnostic).	
CDS View	Declarative models, Fiori/OData, analytics (preferred).	
AMDP		Complex procedural logic (HANA-specific, very powerful).

Q23. What is the difference between AMDP Procedure and AMDP Function? When do you use each?

AMDP Procedure (BY DATABASE PROCEDURE)

- Returns data via EXPORTING parameters.
- Can have multiple in/out parameters.
- Called from ABAP like a normal method.
- SQLScript body uses assignment: out_tab = SELECT ... ;
- Use when: Returning multiple tables, complex multi-step logic.

Example:

```
METHOD my_proc BY DATABASE PROCEDURE
  FOR HDB LANGUAGE SQLSCRIPT USING vbak vbap.
  et_orders = SELECT vbeln, kunnr
    FROM vbak WHERE erdat = :iv_date;
ENDMETHOD.
```

AMDP Function (BY DATABASE FUNCTION)

- Returns a single table (RETURNING).
- Used in CDS as a Table Function.
- Integrated into CDS: define view as select from <table function>.
- Use when: Complex data needed in CDS but cannot be computed declaratively.

Example:

```
METHOD my_func RETURNING VALUE(rt_result)
  TYPE tt_result BY DATABASE FUNCTION
  FOR HDB LANGUAGE SQLSCRIPT USING mara.
  RETURN SELECT matnr, mtart FROM mara
    WHERE mtart = :iv_type;
ENDMETHOD.
```

CDS Table Function consuming AMDP Function:

```
@AbapCatalog.sqlViewName: 'ZTF_VIEW'
define view ZCds_TableFunction as select from zcl_amdp_class->my_func( iv_type: 'FERT' )
{ matnr, mtart }
```


SAP ABAP & ABAP ON HANA - INTERVIEW PREPARATION GUIDE

MNC READY
2024 EDITION

50 QUESTIONS | BASICS TO 4 YEARS EXPERIENCE | TOP MNC PREPARATION | 2024 EDITION

Topics: Data Dictionary • RICEWF • Module Pools • OOP ABAP • CDS • AMDP • Performance Tuning • ABAP 7.5 Syntax

SECTION 7: PERFORMANCE TUNING IN ABAP

Q24. What are the key performance tuning principles in ABAP? Explain with examples.

Answer:

1. **Select Only What You Need:**

- Never use SELECT *. Always specify required fields.

"Bad: SELECT * FROM mara INTO TABLE lt_mara.

"Good: SELECT matnr makt meins FROM mara INTO TABLE lt_mara WHERE makt = 'FERT'.

2. **Use WHERE Clause (Push Conditions to DB):**

- Always filter at the database level, not in ABAP loops. DB can use indexes efficiently.

3. **Avoid Nested SELECT (N+1 Problem):**

- Never select inside a LOOP. Use JOINS or FAE.

"Bad: LOOP AT lt_orders. SELECT SINGLE * FROM kna1 WHERE kunnr = lt_orders-kunnr. ENDLOOP.

"Good: SELECT kunnr name1 FROM kna1 INTO TABLE lt_kna1 FOR ALL ENTRIES IN lt_orders WHERE kunnr = lt_orders-kunnr.

4. **FOR ALL ENTRIES (FAE) Best Practices:**

- Always check that the driver table is NOT initial. FAE with empty table = full table scan!

IF lt_orders IS NOT INITIAL.

SELECT ... FOR ALL ENTRIES IN lt_orders WHERE ... ENDIF.

5. **Use Internal Table Operations:**

- Use SORT + READ TABLE ... BINARY SEARCH or use HASHED TABLE for key-based access.

6. **Buffering:**

- Use table buffering (single/generic/full) for small, frequently-read configuration tables in SE11.

7. **HANA-Specific: Push to DB:**

- Use CDS views / AMDP for aggregations, joins, computations - let HANA do the heavy lifting.
- Avoid SELECT with ORDER BY if sorting can be done in ABAP for small result sets; for large sets, DB sort is faster.



Q26. What tools do you use to analyze ABAP performance? Explain SAT, SCI, and ST05.

Answer:

1. **SAT - ABAP Runtime Analysis (formerly SE30):**

- Profiles program runtime - shows statements, taking most time.
- Hit list sorted by gross time, net time, no. of calls.
- Drill down to DB calls, subroutines, FM calls.
- Use: Run SAT → Execute program → Analyze hit list.

2. **ST05 - SQL Trace / Performance Trace:**

- Records all DB calls made by an ABAP program.
- Shows SQL statement, table, duration, no. of records, indexes.
- Use for: Missing WHERE, SELECT*, index usage, N+1 queries.
- Steps: ST05 → Activate → Run → Deactivate → Display Trace.

3. **SCI - ABAP Code Inspector:**

- Checks coding standards & potential issues.
- Checks: SELECT *, FAE empty check, hardcoded values, obsolete statements, SY-SUBRC not checked, etc.
- Run on programs/packages. Integrated with ATC.

4. **ATC - ABAP Test Cockpit:**

- Successor to SCI. Runs code quality, performance, security checks before transport.

5. **SE30 (older) / Performance Trace Additions:**

- HANA-specific: Use HANA Studio / HANA Cockpit to analyze SQLScript, CDS execution plans, AMDP performance.

6. **SM50 / SM66:**

- Work process overview - shows running programs, identify long-running processes.

Tip:

Use right tool for right problem! ☆

Q28. Explain the VALUE operator. How is it used with structures and internal tables?

Answer:

• **Elementary Types**

DATA(lv_count) = VALUE i(). "0"

DATA(lv_name) = VALUE string(). "empty"

• **Structures**

DATA(lv_address) = VALUE ty_address(street = 'MG Road' city = 'Chennai' country = 'IN' zip = '600005').

• **Append with BASE**

lv_items = VALUE lv_items(BASE lv_items (matnr = 'MAT4' qty = 8 sum = 'EA').

• **VALUE with FOR**

DATA(lv_filtered) = VALUE lt_output(FOR ls IN lt_source WHERE (qty > 5) (matnr = lv_matnr amount = ls_qty * ls_price).



Q25. What is the difference between READ TABLE with BINARY SEARCH vs HASHED TABLE vs SORTED TABLE?

Answer:

1. **Standard Table (default):**

- Linear access - searches from row 1 to n ($O(n)$ complexity).
- Good for: Sequential processing, small tables, frequent inserts.
- READ TABLE lt_std WITH KEY field = val.
 - Linear search, slow for large tables.

2. **Sorted Table:**

- Always maintained in sorted order by defined key.
- Binary search is automatic for key fields.
- $O(\log n)$ access for key reads. Inserts slower than standard.
- READ TABLE lt_sorted WITH TABLE KEY field = val.
 - Binary search automatic.

DATA: lt_sorted TYPE SORTED TABLE OF ty_line WITH UNIQUE KEY matnr werks.

3. **Hashed Table:**

- Uses hash algorithm - $O(1)$ constant time for full key lookup.
- Fastest for key-based reads but no sequential sorting.
- No LOOP AT ... FROM ... TO. Cannot be sorted.
- Best for: Lookup tables, de-duplication, full key access.

DATA: lt_hash TYPE HASHED TABLE OF ty_line WITH UNIQUE KEY matnr.

READ TABLE lt_hash WITH TABLE KEY matnr = lv_matnr INTO ls_line. " $O(1)$

4. **Binary Search on Standard Table:**

- Manual: Sort table first, then READ TABLE ... BINARY SEARCH.
- Good when you need both sequential and keyed access.

SORT lt_std BY matnr.

READ TABLE lt_std WITH KEY matnr = lv_matnr BINARY SEARCH INTO ls_line.

Performance Recommendation:

- Hashed Table: Best for key-only lookups (unique key).
- Sorted Table: Best when you need sorted access + binary search.
- Standard + BINARY SEARCH: When both sequential and keyed access are needed.



SECTION 8: ABAP 7.4/7.5 NEW SYNTAX FEATURES

Q27. What are the key new syntax features in ABAP 7.4 and 7.5? Give examples.

Answer:

1. **Inline Declarations (DATA(...))**

2. **Inline String Templates ({...})**

3. **Constructor Expressions - NEW**

4. **Constructor Expressions - VALUE**

5. **Constructor Expressions - CORRESPONDING**

6. **Table Expressions**

7. **REDUCE**

8. **FILTER**

9. **CONV**

10. **COND and SWITCH**

```
1. SELECT SINGLE * FROM mara INTO @DATA(lv_mara).
2. DATA(lv_mag) = |Material { lv_mara-matnr }
   has type { lv_mara-matnr }.
3. DATA(lv_obj) = NEW obj_material( lv_matnr = '100' ).
4. DATA(lv_line) = VALUE ty_line( matnr = '100'
   werks = '1000' ).
   DATA(lv_tab) = VALUE lt_type( ( matnr = '100' ).
   ( matnr = '200' ) ).
5. DATA(lv_target) = CORRESPONDING ty_target( lv_source ).
6. DATA(lv_mat) = lv_mara[ matnr = '100' ].
   IF line_exists( lv_mara( matnr = '100' ) ). ... ENDIF.
7. DATA(lv_total) = REDUCE i( INIT sum = 0
   FOR ls IN lt_items NEXT sum = sum + ls-qty ).
8. DATA(lv_fert) = FILTER lv_mara( lv_mara WHERE makt = 'FERT' ).
9. DATA(lv_num) = CONV dectfloat16( '3.14' ).
10. DATA(lv_label) = COND string(
   WHEN lv_langs = 'E' THEN 'English' ELSE 'Other' ).
   DATA(lv_text) = SWITCH string( lv_code
   WHEN '01' THEN 'Active'
   WHEN '02' THEN 'Blocked'
   ELSE 'Unknown' ).
```



Q29. What is the CORRESPONDING operator? How does it handle field mapping?

Answer:

• **Basic**

DATA(lv_target) = CORRESPONDING ty_target(lv_source).

• **With MAPPING**

DATA(lv_target) = CORRESPONDING ty_target(lv_source MAPPING tyt_field = src_field).

• **With EXCEPT**

DATA(lv_target) = CORRESPONDING ty_target(lv_source EXCEPT field1 field2).

• **For Internal Tables:**

DATA(lv_target) = CORRESPONDING lt_target(lv_source).

• **With DEEP**

DATA(lv_deep) = CORRESPONDING DEEP ty_target(lv_source).

Comparison:

MOVE-CORRESPONDING: Statement, only matching names.
CORRESPONDING: Expression, supports MAPPING, EXCEPT, DEEP. Usable inline in expressions.



Note: Reflection-based - for huge data, explicit assignment may be faster.

Q30. Explain String Templates ({...}) and string operations in ABAP 7.4.

Answer:

• **String Template**

DATA(lv_mag) = |Hello { lv_name }, your order { lv_orderno } is confirmed!.

• **Formatting in {}**

- Numbers { lv_amount WIDTH = 10 DECIMALS = 2 }.
- Date { ty-date DATE = ISO } → '2004-01-15'.
- Time { ty-zeit TIME = ISO } → '14:30:00'.
- Case { lv_text CASE = UPPER }.
- Alpha { lv_matnr ALPHA = OUT }.

• **Chaining**

DATA(lv_full) = |First| && |second| && |lv_name|.

• **Multi-line**

DATA(lv_html) = |<root><n| &&|
| lv_value |</n>| &&|</root>|.



SAP ABAP & ABAP on HANA - INTERVIEW PREPARATION GUIDE

MNC READY
2024 EDITION

50 QUESTIONS | BASICS TO 4 YEARS EXPERIENCE | TOP MNC PREPARATION | 2024 EDITION

Topics: Data Dictionary • RICEWF • Module Pools • OOP ABAP • CDS • AMDP • Performance Tuning • ABAP 7.5 Syntax

SECTION 9: ADVANCED ABAP CONCEPTS

Q31. What is ALV (ABAP List Viewer)? Compare ALV Grid (CL_GUI_ALV_GRID) vs SALV (CL_SALV_TABLE).

Answer:

ALV (ABAP List Viewer) is SAP's standard framework for displaying tabular data in an interactive, user-friendly grid with built-in sort, filter, export, and layout features.

CL_GUI_ALV_GRID (Classic ALV Grid)	CL_SALV_TABLE (SALV - Simple ALV)
- Uses GUI controls framework. Requires container (CL_GUI_CUSTOM_CONTAINER or CL_GUI_DOCKING_CONTAINER) and a screen with custom control area. - Very flexible - modify cells, register events (button click, hotspot, double-click, toolbar), change cell colors, enable cell-level editing. - More complex implementation but gives maximum control. Good for: Full CRUD screens, editable ALV, complex toolbar, cell-level highlighting.	- Simpler API. No need for screen or container - can display in popup or fullscreen. - Read-only by default (no cell editing). - Event handling via event objects, not custom modules. - Less flexible but much faster to develop for display-only reports. SALV Example: <pre>DATA(lv_salv) = cl_salv_table=>factory(IMPORTING +lv_salv_table = lv_salv CHANGING +table = lt_data). lv_salv->set_optimize(). lv_salv->display().</pre>

Field Catalog:

For CL_GUI_ALV_GRID, define FIELDCAT (LVC_T_FCAT) to describe each column: field name, label, output length, data type, color, hotspot flag, edit flag.

SALV auto-generates fieldcat from table structure but allows post-modification.

- MNC Preference: Use SALV for new read-only reports (faster development). Use CL_GUI_ALV_GRID for editable grids or complex event-driven scenarios.

Q32. What is RFC (Remote Function Call)? Explain different RFC types.

Answer:

RFC (Remote Function Call) is SAP's protocol for calling function modules on a remote SAP system (or external system). Backbone of SAP system integration (BAPIs, IDocs, PI/PO, SAP2SAP etc.).

RFC Types:

- trRFC (Synchronous RFC):** Standard RFC. Caller waits for response. Both systems must be available. Used for: BAPIs, real-time exchange.
- arRFC (Asynchronous RFC):** Fire-and-forget call. Caller does not wait. Called FM runs independently. Good for parallel processing.
- trRFC (Transactional RFC):** Ensures exactly-once execution. Queued in ARFCSTATE table. Used for IDoc outbound processing.
- qRFC (Queued RFC):** Extension of trRFC. Calls queued in named inbound/outbound queues. Guarantees sequence & exactly-once. Used in ALE/EDI.
- bgRFC (Background RFC):** Modern replacement for trRFC & qRFC. Better performance, monitoring in SDBGFCMON.
- Two variants: Unit queue (trRFC replacement), Supervisor queue (qRFC replacement).

RFC Function Module Requirements:

- Must be set as 'Remote-Enabled Module' in FM attributes.
- Parameters: IMPORTING, EXPORTING, CHANGING, TABLES (legacy), EXCEPTIONS.
- No pass by reference - all parameters passed by value.

CALL FUNCTION Syntax:

```
CALL FUNCTION 'Z_MY_REMOTE_FM' DESTINATION 'RFC_DESTINATION'
  EXPORTING
    lv_input = lv_val
  IMPORTING
    ex_result = lv_result
  EXCEPTIONS
    system_failure = 1
    communication_failure = 2
    OTHERS = 3.
```

Q36. What is the difference between MOVE-CORRESPONDING and CORRESPONDING in ABAP 7.5?

Answer:

- MOVE-CORRESPONDING:** Procedural statement. Copies matching-named fields from source struct to target struct. Cannot be used inline or in expressions.
- CORRESPONDING:** Functional constructor expressions (7.4+). Can be used inline, supports MAPPING, EXCEPT, DEEP, BASE. More versatile and readable.

Old (MOVE-CORRESPONDING):
MOVE-CORRESPONDING lv_src
TO lv_high.

New (CORRESPONDING):
lv_high = CORRESPONDING lv_target(lv_src.
MAPPING lv_high -> lv_low).
EXCEPT lv_high -> lv_low).

Q33. What is a BAPI? How do you test and call a BAPI?

Answer:

BAPI (Business Application Programming Interface) is a standardized, stable RFC-enabled FM that provides official interface to SAP business objects. Used to create/change/read data programmatically.

BAPI Naming Convention:

- BAPI_<OBJECT>_<METHOD>

Eg. BAPI_MATERIAL_SAVEDIRECT, BAPI_PO_CREATE1, BAPI_SALESORDER_CREATEFROMDAT2

Standard BAPI Parameters:

- RETURN table (BAPIRET2):** Contains messages (SAP/AVI, message ID, number, text). Always check RETURN.
- COMMIT:** Most BAPIs do NOT commit automatically. Call BAPI_TRANSACTION_COMMIT explicitly after success.

Calling a BAPI in ABAP:

```
DATA: lt_return TYPE TABLE OF BAPIRET2.
CALL FUNCTION 'BAPI_PO_CREATE1'
  EXPORTING
    pheader = lv_header
  TABLES
    pitem = lt_items
  RETURN = lt_return.

* Check for errors
READ TABLE lt_return WITH KEY type = 'E' TRANSPORTING NO FIELDS.
IF sy-subrc <> 0
  CALL FUNCTION 'BAPI_TRANSACTION_COMMIT' EXPORTING wait = 'X'.
ELSE
  CALL FUNCTION 'BAPI_TRANSACTION_ROLLBACK'.
ENDIF.
```

Testing a BAPI:

Use SE37 → Enter BAPI name → F8 to test. Or use BAPI transaction for business-object-oriented view. Check RETURN table for messages.

BAPI Finder:

Transaction BAPI or SE37 with search.

BOR (Business Object Repository) shows all available BAPIs per business object type.

Q37. What is Enhancement Framework? Explain Implicit and Explicit Enhancement Points.

Answer:

Enhancement Framework (EF) allows adding custom code to standard SAP without modifying it - upgrade-safe customization.

1. Implicit Enhancement Points:

- SAP provides enhancement spots automatically in every program: start/end of programs, FMs, subroutines, methods, between statements.
- Activate via SE80/ADT → right-click → 'Enhancement Operations' → 'Add Enhancement'.
- Invisible in standard source - shown only if enhancements exist.

2. Explicit Enhancement Points (ENHANCEMENT-POINT):

- SAP marks places: ENHANCEMENT-POINT (name) SPOTS (spotname).
- Implement via SE80 → Enhancement Spots.

3. Enhancement Sections:

- Mark a range that can be REPLACED: ENHANCEMENT-SECTION ... END-ENHANCEMENT-SECTION.
- Your implementation completely replaces the section. (Points only add code).

Composite Enhancement Spots: Group multiple spots into named spot object for easier management (SE80).

Q34. Explain the ABAP LUW (Logical Unit of Work) and how SAP manages database consistency.

Answer:

LUW is a sequence of DB operations (INSERT, UPDATE, DELETE) that must either all succeed or all fail. Bounded by COMMIT WORK and ROLLBACK WORK.

DB LUW vs SAP LUW:

- DB LUW:** Database transaction. From last COMMIT to next COMMIT. Screen change does NOT commit.
- SAP LUW:** Higher-level concept spanning multiple dialog steps (screens, RFC calls). Uses Update Tasks to bundle DB changes and commit atomically.

Update Task (V1 and V2):

- CALL FUNCTION IN UPDATE TASK:** Registers FM to run in update work process asynchronously when COMMIT WORK is called.
- V1 (Synchronous):** Critical DB ops. Called first. If V1 fails, update document set to error.
- V2 (Asynchronous):** Less critical ops (stats, derived data). Called after V1 completes.

```
CALL FUNCTION 'Z_UPDATE_STOCK' IN UPDATE TASK
  EXPORTING lv_mater = lv_mater lv_qty = lv_qty.
COMMIT WORK.

* Triggers update task FM and DB commit
COMMIT WORK AND WAIT.

* Wait for update task to complete before continuing.
ROLLBACK WORK.

* Cancels pending DB changes and update task calls.
```

Q35. What is SAP Memory, ABAP Memory, and Shared Memory? How do you use them?

Answer:

1. ABAP Memory (EXPORT/IMPORT):

- Local to single internal session (call stack).
- Used via EXPORT TO MEMORY ID / IMPORT FROM MEMORY ID.

```
EXPORT lv_value TO MEMORY ID 'ZKEY'.
IMPORT lv_value FROM MEMORY ID 'ZKEY'.
```

- Freed at end of session or by FREE MEMORY ID 'ZKEY'.

2. SAP Memory (SET/GET PARAMETER = SPA/GPA):

- Shared across all sessions for a user in a login. Persists across transactions.
- Used to pre-fill screen fields with user defaults.

```
SET PARAMETER ID 'WRK' FIELD lv_works.
GET PARAMETER ID 'WRK' FIELD lv_works.
```

- Parameter IDs in DDIC (SU20/SU21).

Std IDs: BUK (Company Code), WRK (Plant), KUN (Customer).

3. Shared Memory Objects (SHM):

- Shared across all user sessions on same app server. Implemented using ABAP Shared Objects.
- Used for: reference data caching (currency rates, material descriptions, org hierarchy etc.).
- Define area class → attach/lock → read/write data.

Cluster DB / Shared Buffer: SAP also has internal buffers (program buffer, CUA buffer, nametab buffer) to speed up repeated reads.

Q38. How do you read and create data using ABAP 7.5 inline SELECT syntax?

Answer:

- Single record inline:**
SELECT SINGLE matnr matnt matns FROM mats
INTO @DATA(lv_mats) WHERE matnr = @lv_mater.
- Internal table inline:**
SELECT matnr matnt matns FROM mats
INTO TABLE @DATA(lt_mats) WHERE matnt = 'PERT'.
- Expanded host variables (H):**
SELECT matnr FROM mats INTO TABLE @DATA(lt_mats).
WHERE works = @lv_works AND matnt IN @lt_matnt_range.
- Aggregation inline:**
SELECT COUNT(*) FROM mats INTO @DATA(lv_count) WHERE matnt = 'RON'.
- Join inline:**
SELECT a-matnr a-matnt b-works b-label FROM mats AS a
INNER JOIN mats AS b ON a-matnr = b-matnr
INTO TABLE @DATA(lt_result) WHERE a-matnt = 'PERT' AND
b-works = '1000'.
- FOR ALL ENTRIES inline:**
IF lv_matk IS NOT INITIAL.
SELECT vmatn pair matnr FROM shap INTO TABLE @DATA(lt_shap)
FOR ALL ENTRIES IN @lv_matk WHERE shap = @lv_matk-shap.
ENDIF.
- GROUP BY with ORDER BY:**
SELECT matnt COUNT(*) AS cnt FROM mats
INTO TABLE @DATA(lt_summary) GROUP BY matnt
ORDER BY cnt DESCENDING.

**Q39. What is RAP (RESTFUL ABAP Programming Model)?
How is it related to CDS and OData?**

Answer:

RAP (RESTful ABAP Programming Model) is SAP's modern framework (introduced with SAP BTP ABAP Environment and S/4HANA) for building Fiori applications and OData services following REST principles. It supersedes the combination of SEGW + CDS + Manual OData implementation.

RAP Stack Components:

1. **CDS Root View (Behavior Definition):**
Define the Business Object structure and hierarchy (root, child, grandchild).
2. **Behavior Definition (BDEF):**
Declare what operations the BO supports: create, update, delete, actions, validations, determinations.
Define managed vs unmanaged implementation.
3. **Behavior Implementation Class (BIL):**
ABAP class implementing FOR BEHAVIORS.
Contains actual logic for validations, determinations, actions.
4. **Service Definition:**
Exposes CDS views and operations as OData entities.
5. **Service Binding:**
Binds service to OData V2 / V4 protocol.
Activates as OData endpoint. Can preview in Fiori Preview.

RAP Behavior Types:

- **Managed:** SAP framework automatically handles CRUD persistence to DB tables. You only code validations, determinations, actions.
- **Unmanaged:** You code all CRUD logic (save sequence) yourself. More control, more code.

RAP vs Classic SEGW:

- **SEGW:** Manual OData service building, separate from CDS. Complex to maintain, no built-in draft handling.
- **RAP:** CDS-centric, automatic OData generation, built-in draft/lock handling, SAP Fiori elements compatible.

MNC Relevance:

RAP is mandatory knowledge for S/4HANA Fiori development. In MNC interviews for S/4HANA roles, expect questions on BDEF syntax, managed vs unmanaged, actions vs validations vs determinations. Know CDS view types, behavior definitions, and service binding concepts well.

Q40. Explain the concept of Table Buffering in SAP and when would you enable it?

Answer:

Table Buffering: allows frequently-read, rarely-changed database tables to be loaded into the SAP application server's shared memory (table buffer). Subsequent reads are served from memory, avoiding database roundtrips - dramatically improving performance for lookup tables.

Buffering Types (set in SE11, Table Technical Settings):

- **No Buffering:** All reads go directly to database.
Use for tables with frequent changes.
- **Single Record Buffering:** Only specific, explicitly read records are buffered. Used when accessing a few specific keys frequently.
- **Generic Area Buffering:** Records are buffered based on a key prefix (first N fields of key). Useful for org-structure-keyed tables.
- **Full Table Buffering:** Entire table loaded into buffer on first access. Best for small, static tables (10K records max recommended).

When to Enable Buffering:

- Small table size (fits in memory comfortably)
- Read-frequently, write-rarely (configuration/customizing data, currency codes, country/language tables)
- Non-critical data (slight staleness acceptable - buffer syncs on change but not immediately across all app servers)

Buffer Synchronization:

On COMMIT WORK after a change to a buffered table, SAP sends an invalidation request to all application servers. Each server re-reads from DB on next access. There's a short window of staleness across servers.

Bypassing the Buffer:

```
SELECT ... FROM ztable BYPASSING BUFFER WHERE ...  
"Forces direct DB read"
```

Examples of buffered standard tables:

- T001 (Company Codes) - Full Buffering
- T006 (Units of Measure) - Full Buffering
- T005 (Countries) - Full Buffering

Q41. What is the difference between AUTHORITY-CHECK and permission objects? How do you implement authorization in ABAP?

Answer: SAP Authorization is a security framework that controls what data and operations a user can access. Authorization Objects define the fields and values that are checked. AUTHORITY-CHECK is the ABAP statement that performs the check.

• Authorization Object:

- Defined in SU21. Contains up to 10 authorization fields (e.g., BUKRS for company code, ACTVT for activity type).
- Standard objects: M_MSEG_WMB (goods movement/plant), S_TCODE (transaction code access), K_CCA (cost center).

• AUTHORITY-CHECK Statement:

```
AUTHORITY-CHECK OBJECT 'M_MSEG_WMB'
  ID 'WERKS' FIELD lv_werks
  ID 'ACTVT' FIELD '03'. "01=Create, 02=Change,
                        03=Display, 06=Delete
  IF sy-subrc <> 0.
    MESSAGE 'No authorization' TYPE 'E'.
  ENDIF.
```

• Activity Values (ACTVT):

01 = Create, 02 = Change/Update, 03 = Display, 06 = Delete, 16 = Execute, 23 = Maintain

• Authorization in Reports:

Use AUTHORITY-CHECK at the start of the program or before accessing sensitive data. For ALV, use CL_GUI_ALV_GRID field catalog 'NO_AUTH' or implement your own field-level authorization.

• Profile Generator (PFCG):

Roles are created in PFCG and assigned to users in SU01. Each role contains authorization objects with specific field values. SU53 shows the last failed authorization check for debugging.

Q44. What is a Function Group and how does it differ from a Class? When would you use a Function Group?

Function Group:

- A container (namespace) for related Function Modules. All FMs in a group share the same global data (declared in FUNCTION-POOL / TOP include).
- Created in SE80 / SE37. Has includes: TOP (global data), Lxxx (source for FMs), LxxxUxx (includes for specific FMs).
- When any FM from a group is called, the entire function group is loaded into memory (including global data). All subsequent FMs in that group within the same session share that loaded state.
- Used for: RFC-enabled modules (BAPI), SAP standard exit functions, BDC session handling, legacy integration code.

Class vs Function Group:

Aspect	Function Group	Class
Paradigm	Procedural	OOP
Data	Global data shared across all FMs	Attributes scoped to instance/class
Instance	Single instance per session	Multiple instances possible
Features	No inheritance, no polymorphism	Full OOP (inheritance, polymorphism, interfaces, events)
RFC enabled	Yes (FMs can be RFC-enabled)	No (Classes cannot be RFC-enabled)

When to Use Function Groups:

- Creating RFC-enabled Function Modules for BAPI-style interfaces (classes cannot be RFC-enabled directly).
- Maintaining existing legacy Function Group code.
- Screen exits, BADI implementations that require FM structure (classic BADI).

MNC Recommendation: For all new development in S/4HANA, prefer ABAP Classes. Function Groups are still needed for RFC/BAPI interfaces, but encapsulate the business logic in classes and call from the FM.

Q42. What are the different ways to call SAP programs and pass data between them?

1. CALL TRANSACTION:

CALL TRANSACTION 'MM03' AND SKIP FIRST SCREEN. "With BDC data

- Calls a dialog transaction. Data passed via SY-TCODE, BDC data, or SUBMIT (for report transactions).

2. SUBMIT:

```
SUBMIT zmy_report WITH p_werks = '1000'
  WITH s_matnr IN lv_matnr AND RETURN.
```

- Calls a report program (Type 1). AND RETURN comes back to calling program after execution.
- SUBMIT ... EXPORTING LIST TO MEMORY: Captures output in memory for retrieval.

3. CALL FUNCTION (RFC/Local):

```
CALL FUNCTION 'Z_MY_FM'
  EXPORTING iv_input = lv_val
  IMPORTING ev_result = lv_res.
```

4. Memory Transfer (EXPORT/IMPORT):

```
EXPORT ls_data TO MEMORY ID 'ZKEY'. "In program A (same session)
IMPORT ls_data FROM MEMORY ID 'ZKEY'. "In program B
```

5. SET/GET PARAMETER (SPA memory):

```
SET PARAMETER ID 'BUK' FIELD '1000'.
GET PARAMETER ID 'BUK' FIELD lv_bukrs.
```

6. Object Method Calls:

- Instantiate class and call methods. Pass complex objects via reference (TYPE REF TO).

7. Shared Objects:

- Cross-session data sharing via ABAP Shared Memory Objects (for reference data caching).

8. ABAP Messaging (RFC/Events):

- Use RAISE EVENT ... with EXPORTING parameters for event-driven communication between objects.

Q43. Explain how you would debug a complex ABAP issue in production. What tools do you use?

Answer: Debugging in production (or production-like environments) requires careful, non-intrusive analysis. SAP provides multiple tools for this purpose.

- ABAP Debugger (SE80 / ADT):** Set breakpoints (/h), external breakpoints for background/RFC. Step Over (F6), Step Into (F5), Step Out (F7), Continue (F8).
- SAT (Runtime Analysis):** Profile without stopping execution. Identifies hotspot functions/statements. Use for performance issues.
- ST05 (SQL Trace):** Capture and analyze all DB SQL executed. Identify slow queries, missing indexes, SELECT *.
- SLG1 (Application Log):** Read logs written by ABAP programs using BAL* function modules. Always check SLG1 for application-level logs.
- SM21 (System Log):** System-level errors - short dumps, RFC errors, update task failures.
- ST22 (ABAP Short Dumps):** Analyze runtime errors (DUMP analysis). Shows exact code line, variable values at time of dump.
- SM37 (Background Job Log):** For background job issues - check job log for error messages, spool output.
- SM13 (Update Errors):** Check for failed update task executions (V1/V2).

Best Practice: Never set interactive breakpoints in production - they block the work process. Use external breakpoints with user restriction, or add temporary logging (SLG1) and analyze logs. Reproduce in DEV/QA first.

Q45. Explain the concept of ABAP Unit Testing. How do you write unit tests in ABAP?

Answer: ABAP Unit is SAP's built-in unit testing framework (similar to JUnit for Java). It allows writing automated tests for ABAP code that run independently, verify expected behavior, and catch regressions.

Test Class Structure:

```
CLASS zcl_my_test DEFINITION FINAL FOR TESTING
  DURATION SHORT RISK LEVEL HARMLESS.
  PRIVATE SECTION.
    METHODS: setup,
              teardown,
              test_calculate_tax FOR TESTING.
ENDCLASS.
```

@Test class must be final for testing only

CLASS zcl_my_test IMPLEMENTATION.

```
METHOD setup. "Runs before each test
  "Initialize test fixtures
ENDMETHOD.

METHOD test_calculate_tax.
  DATA(lo_calc) = NEW zcl_tax_calculator( ).
  DATA(lv_tax) = lo_calc->calculate( iv_amount = 1000 iv_rate = 18 ).
  cl_abap_unit_assert=>assert_equals(
    exp = 180 act = lv_tax msg = 'Tax calculation failed' ).
ENDMETHOD.
ENDCLASS.
```

Key Assertions (CL_ABAP_UNIT_ASSERT):

- assert_equals: Check expected vs actual value.
- assert_not_initial: Verify a variable is populated.
- assert_true / assert_false: Check boolean conditions.
- assert_subrc: Check SY-SUBRC value.
- fail: Explicitly fail a test with a message.

Test Doubles / Mocking:

Use interfaces for dependencies so test doubles can be injected. The ABAP Test Double Framework (CL_ABAP_TESTDOUBLE) generates mock objects from interfaces automatically - configure method stubs and verify calls.

Running Tests:

In ADT (Eclipse): Right-click class -> Run As -> ABAP Unit Test.
In SAP GUI: SE80 -> Program -> Execute -> ABAP Unit Test.
Results shown as green/red in test runner.

Q46. What are ABAP Events? How are events declared and handled in OOP ABAP?

Answer: ABAP Events are a publish-subscribe mechanism in OOP ABAP. A class can raise (publish) events, and other classes register handlers to respond to those events. This enables loose coupling between components.

1. Declaring Events:

```
CLASS zcl_order DEFINITION.
  PUBLIC SECTION.
    EVENTS: order_saved
            EXPORTING VALUE(ev_orderno) TYPE vbeln.
ENDCLASS.
```

2. Raising Events:

```
CLASS zcl_order IMPLEMENTATION.
  METHOD save_order.
    "... save logic ...
    RAISE EVENT order_saved
      EXPORTING ev_orderno = lv_orderno.
  ENDMETHOD.
ENDCLASS.
```

3. Handler Class:

```
CLASS zcl_notifier DEFINITION.
  PUBLIC SECTION.
    METHODS: on_order_saved
            FOR EVENT order_saved OF zcl_order
            IMPORTING ev_orderno.
ENDCLASS.
```

```
CLASS zcl_notifier IMPLEMENTATION.
  METHOD on_order_saved.
    WRITE: / |Order { ev_orderno }
      was saved. Sending notification. |
  ENDMETHOD.
ENDCLASS.
```

4. Registering the Handler:

```
DATA: lo_order TYPE REF TO zcl_order,
      lo_handler TYPE REF TO zcl_notifier.

lo_order = NEW zcl_order( ).
lo_handler = NEW zcl_notifier( ).
SET HANDLER lo_handler->on_order_saved FOR lo_order.
lo_order->save_order( ). "Triggers event
```

Static Events: Events can be CLASS-EVENTS (class-level), raised with RAISE EVENT from static method, and registered with SET HANDLER ... FOR ALL INSTANCES to handle events from any instance.

SAP ABAP & ABAP ON HANA - INTERVIEW PREPARATION GUIDE

50 QUESTIONS | BASICS TO 4 YEARS EXPERIENCE | TOP MNC PREPARATION | 2024 EDITION

MNC READY
2024 EDITION

Topics: DDIC • RICEWF • Module Pools • OOP ABAP • CDS • AMDP • Performance • ABAP 7.5 Syntax

Q47. What is the difference between SELECT SINGLE and SELECT UP TO 1 ROWS?

A) SELECT SINGLE

- Designed to read exactly one record - expects WHERE clause to specify the full primary key (or unique index).
- IF WHERE does NOT uniquely identify one record, DB returns an arbitrary record.
- Generates DB SELECT with ROWNUM = or equivalent.
- Fast for key-based lookups.
- SY-SUBRC = 0 if found, 4 if not found.

```
SELECT SINGLE matnr mtrt
FROM mara INTO @DATA(ls_mara)
WHERE matnr = @lv_matnr.
```

B) SELECT ... UP TO 1 ROWS

- Reads UP TO 1 record from a multi-record result set.
- Can have non-unique WHERE conditions.
- Returns the first record found (based on DB physical order unless ORDER BY is specified).
- Result goes into internal table.
- Check if table has entries.

```
SELECT matnr mtrt FROM mara
INTO TABLE @DATA(it_mara)
UP TO 1 ROWS
WHERE mtrt = 'FERT'.
ORDER BY matnr.

IF it_mara IS NOT INITIAL.
  DATA(ls_first) = it_mara[1].
ENDIF.
```

Key Difference:

- SELECT SINGLE**: For full-key (unique) reads. Assumes uniqueness - undefined behavior if condition matches multiple rows.
- UP TO 1 ROWS**: Explicitly limits result set size. Use when you want the 'first' record from a non-unique condition (esp. with ORDER BY).

Best Practice:

Use **SELECT SINGLE** only when WHERE clause references the full primary key or a unique secondary index. Use **UP TO 1 ROWS** with ORDER BY when you need the first record by a specific ordering from non-unique condition.

Q49. What are the ABAP Clean Code principles and how do they apply to S/4HANA development?

Answer: ABAP Clean Code (SAP's open-source Clean ABAP style guide) makes code readable, maintainable & testable.

Key Principles:

- Use meaningful names: `lv_material_number` not `lv_x`.
- Prefer ABAP 7.4+ syntax: `DATA(...)`, `VALUE(...)`, `CORRESPONDING`, string templates over old `CLEAR + MOVE`.
- Small methods: Each method does ONE thing (~30 lines).
- Avoid magic numbers: Use constants or named values.
CONSTANTS: `c_activity_display TYPE activ_auth VALUE '03'`.
- Handle errors with exceptions: Use `TRY-CATCH-ENDTRY`.
- Avoid obsolete statements: No `MOVE`, `COMPUTE`, old `TYPES`. Use modern arithmetic & type system.
- Guard clauses early exit: Return/raise early to avoid deep nesting.
IF `lv_amount <= 0`, `RAISE EXCEPTION TYPE zcx_invald_amount`. `ENDIF`.
- Test-driven: Write unit tests. Design classes to be testable (interfaces, dependency injection, constructor/setter).
- Avoid comments for obvious code: Code should be self-documenting. Use comments for WHY, not WHAT.

MNC Context:

MNCs with ABAP CoE enforce Clean ABAP via ATC checks during transport. Knowing these principles shows maturity beyond just syntax knowledge - a strong differentiator in interviews.

Common Topics to Prepare Real Scenarios For:

- A performance issue you fixed (tool used, root cause, fix)
- An integration you built (IDoc/BAPI/RFC - business need, error handling)
- A complex enhancement (BADI/User exit - standard program, custom logic)

Q48. What are CDS Table Functions and how do you use them with AMDP?

Answer: A CDS Table Function delegates its data retrieval logic to an AMDP function (SQLScript on HANA). It bridges declarative CDS and procedural AMDP, allowing complex HANA-native logic to be consumed via the CDS layer.

When to Use:

- When required data transformation cannot be expressed in CDS.
- When you need HANA-specific SQLScript features (CE, Window functions, SERIES, ML functions).
- For reuse: CDS table function can be consumed by other CDS views, Fiori apps, ABAP Open SQL.

Step 1 - Define AMDP Function in Class

```
CLASS zcl_sales_amdp DEFINITION PUBLIC FINAL CREATE PUBLIC.
PUBLIC SECTION.
  INTERFACES: if_amdp_marker_hdb.
  CLASS-METHODS: get_top_customers
    IMPORTING VALUE(iv_limit) TYPE i.
    RETURNING VALUE(rt_result) TYPE STANDARD TABLE OF
      zty_customer_result WITH DEFAULT KEY.
ENDCLASS.

CLASS zcl_sales_amdp IMPLEMENTATION.
METHOD get_top_customers BY DATABASE FUNCTION FOR HDB
LANGUAGE SQLSCRIPT USING vtab knat.
RETURN SELECT TOP :iv_limit k.kunnr, k.name1, SUM(v.netwr) AS total_sales
FROM knat AS k JOIN vtab AS v ON k.kunnr = v.kunnr
GROUP BY k.kunnr, k.name1 ORDER BY total_sales DESC;
ENDMETHOD.
ENDCLASS.
```

Step 2 - Define CDS Table Function

```
@ClientHandlingType: #CLIENT_INDEPENDENT
define table function ZCds_TopCustomers
with parameters for limit : abap-int4
returns { kunnr : kunnr_d; name1 : name1_gp; total_sales : netwr_kk; }
implemented by method zcl_sales_amdp--get_top_customers;
```

Step 3 - Consume in ABAP

```
SELECT * FROM ZCds_TopCustomers( iv_limit = 10 ) INTO TABLE @DATA(it_top).
```

Q50. Describe a complex ABAP scenario you've worked on and how you approached it. (STAR Method)

S - Situation: Build a custom procurement analytics dashboard in Fiori to show real-time PO aging, vendor KPIs, material availability across 15 plants (~5,000 POs/day).

T - Task: Initial prototype (traditional ABAP with nested SELECTs) took 45 sec. Need sub-5-sec response time.

A - Action:

- Redesigned using layered CDS: Basic (EKKO/EKPO/EKET), Composite with associations (LFA1, MARA), Consumption CDS with @UI & @odata.
- Implemented AMDP Table Functions (SQLScript) for aging calc. & multi-plant inventory aggregation (window functions).
- Used @Analytics.dataCategory: #CUBE for SAC integration.
- Used associations (instead of JOINS) for optional data (contacts, plant details) - lazy loading when user drills down.
- Added ABAP Unit tests for AMDP logic & mock CDS test double.

R - Result: Response time reduced from 45 sec to 2.3 sec. Dashboard live for 3 plants first, then rolled out globally in 6 months.

Key Takeaway:

In MNC interviews, frame answers with clear business context, technical decisions (and WHY) + measurable outcomes. Demonstrates both technical depth & business acumen.

- A CDS / Fiori / RAP implementation for S/4HANA roles
- A migration / conversion project (BDC/LSMW - approach, challenges)